WEEK 02

Relational Algebra and SQL Review Studio

SQL for finding requests, checking reports, and testing a careful data change.

02

From a reporting question to SQL

Which high-priority requests are still active?

QUESTION

One result row should represent one request that still needs work.

CONDITION

Keep priority high and status new, open, or in_progress.

CHECK

Read the matching ticket IDs in the source data as well as the query result.

Result grain: what one row represents

One row per ticket

A ticket row describes one request. Keeping ticket_id identifies it.

Ticket identifiers

One row per event

Ticket 1003 has three events. The event join returns three rows for that one request.

Ticket-event pairs

One row per status

Grouping by status returns one row for each status and its ticket count.

Status groups

Relational operations and SQL

Operation	Meaning	SQL form	Important distinction
Selection σ	Keep rows where a condition is true	WHERE	UNKNOWN does not pass WHERE
Projection π	Choose attributes (columns)	SELECT column list	SQL retains duplicates by default
Join ⋈	Form matching pairs of rows	JOIN ... ON	One source row can have many matches
Grouping γ	Summarize rows into groups	GROUP BY	An extension of basic relational algebra
Difference –	Keep rows in A that are absent from B	EXCEPT	Same column count and compatible types

Chapter 2 develops the notation and the differences between mathematical sets and SQL results.

Filtering rows and choosing columns

```
SELECT ticket_id, subject, priority
FROM metro_support.tickets
WHERE priority = 'high'
ORDER BY ticket_id;
```

metro_support is the schema. tickets is the table.

Reading the result

- Four IDs: 1001, 1005, 1008, 1011.
- Three columns describe each matching ticket.
- ORDER BY changes order, not membership.

Missing assignees and SQL NULL

Predicate

- `assignee_id = NULL`
- `assignee_id IS NULL`
- `assignee_id IS NOT NULL`

Result on the lab data

- No rows. Ordinary equality with NULL does not become true.
- Tickets 1004 and 1009. Their assignees are missing.
- Ten rows. Each has an assignee value.

The three newest active requests

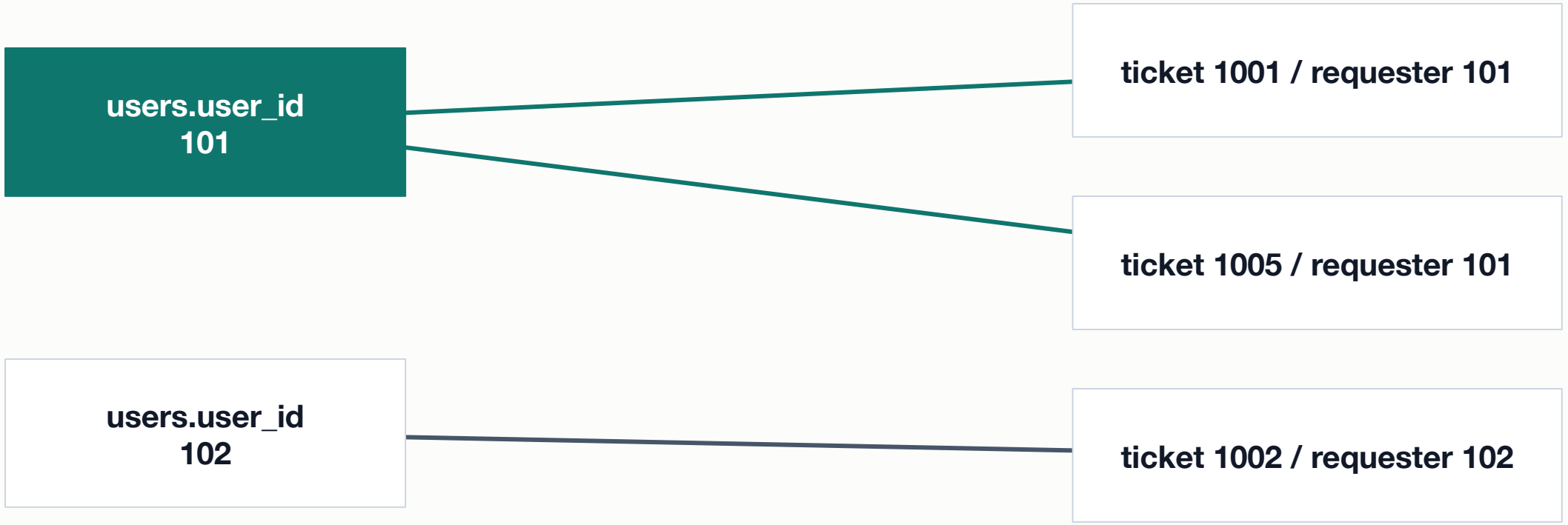
```
SELECT ticket_id, opened_at, priority
FROM metro_support.tickets
WHERE status IN ('new', 'open', 'in_progress')
ORDER BY opened_at DESC, ticket_id DESC
LIMIT 3;
```

Result IDs, in order: 1011, 1009, 1007.

Meaning

- Filter active tickets first.
- Sort newest to oldest.
- Use ticket_id to break ties.

Matching a ticket to its requester



Three-ticket excerpt. Requester 101 (101) requested 1001 and 1005; Requester 102 (102) requested 1002.

A join condition connects related IDs

```
SELECT t.ticket_id, t.subject, u.display_name
FROM metro_support.tickets AS t
JOIN metro_support.users AS u
  ON u.user_id = t.requester_id
WHERE t.status = 'open'
ORDER BY t.ticket_id;
```

requester_id means who reported the issue.
assignee_id means who is handling it.

Read it in this order

- Source tickets and users
- Matching identity condition
- Open-ticket filter and output grain

Lab 1: Requests that need attention

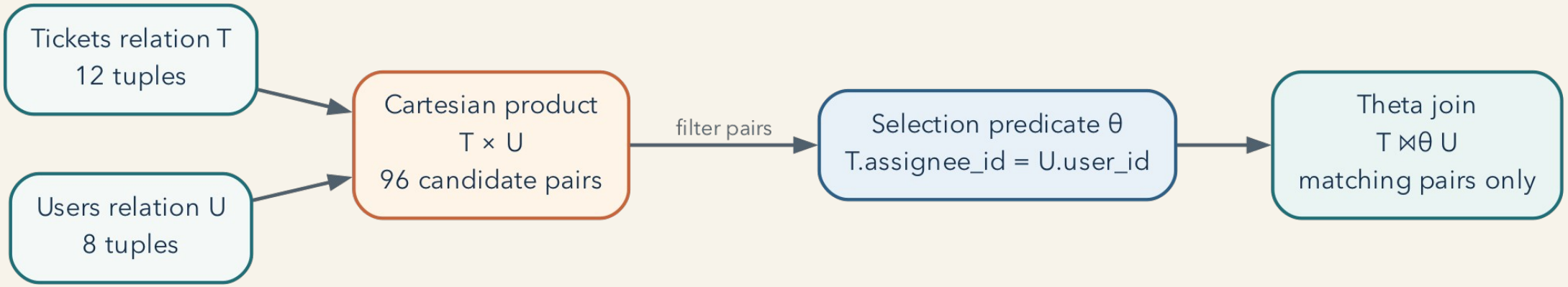
Individual work with the complete fixture: 8 users, 12 tickets, 21 events.

- Write three queries: high/urgent active work, unassigned work, and active Harbor requests.
- Compare = NULL with IS NULL and explain why the results differ.
- Check the Harbor query against the CSVs. Identify one included and one excluded ticket.

Brightspace submission: week_02_sql_review.sql

WORKED VISUAL

An assignee join selects matching pairs




```
tickets.assignee_id = users.user_id
```

Keeping tickets without an assignee

INNER JOIN

- Keeps only matching pairs
- Omits 1004 and 1009. Both have a NULL assignee_id.
- Useful when the relationship must exist for the question

LEFT JOIN

- Returns all 12 tickets when no later filter removes them.
- Produces NULL right-side attributes when unmatched
- Useful for finding unassigned or missing relationships

One ticket can have several events

Ticket 1003 has three events. A join returns three matching pairs.

TICKET

One ticket may have many event tuples.

JOIN ROW

One output row can represent one ticket-event pair.

COUNT

Use the correct grain, grouping, or distinct identity.

Summaries with GROUP BY and HAVING

```
SELECT status, count(*) AS ticket_count
FROM metro_support.tickets
WHERE opened_at >= DATE '2026-02-01'
GROUP BY status
HAVING count(*) >= 2
ORDER BY ticket_count DESC, status;
```

resolved 4, open 3, in_progress 2, new 2.
The closed group has 1 ticket and is omitted.

Result grain

- WHERE filters source ticket rows.
- GROUP BY makes one row per status.
- HAVING keeps qualifying status groups.

Two ways to name the agent relationship

Nested query

```
SELECT ticket_id
FROM metro_support.tickets
WHERE assignee_id IN (
  SELECT user_id
  FROM metro_support.users
  WHERE role = 'agent'
)
ORDER BY ticket_id;
```

Common table expression (CTE)

```
WITH agents AS (
  SELECT user_id
  FROM metro_support.users
  WHERE role = 'agent'
)
SELECT t.ticket_id
FROM metro_support.tickets AS t
JOIN agents AS a
  ON a.user_id = t.assignee_id
ORDER BY t.ticket_id;
```

Both return the same 10 assigned ticket IDs. A CTE names a result for this statement only.

A staff report that includes zero

```
WITH resolved AS (
  SELECT ticket_id, assignee_id
  FROM metro_support.tickets
  WHERE status = 'resolved'
)
SELECT u.user_id, u.display_name,
       count(r.ticket_id) AS resolved_count
FROM metro_support.users AS u
LEFT JOIN resolved AS r
  ON r.assignee_id = u.user_id
WHERE u.role IN ('agent', 'supervisor')
GROUP BY u.user_id, u.display_name
ORDER BY u.user_id;
```

Resolved tickets

Staff	Count
Agent 201 (201)	2
Agent 202 (202)	2
Agent 203 (203)	0

Agent 203 has no match.
COUNT(r.ticket_id) gives 0.
COUNT(*) would give 1.

Combining result sets

Operator	Membership	Duplicates	Example with A = [1, 2], B = [2, 3]
UNION	Rows from either result	Removed	1, 2, 3
UNION ALL	All rows from both results	Retained	1, 2, 2, 3
INTERSECT	Rows in both results	Removed by default	2
EXCEPT	Rows in A absent from B	Removed by default	1

Values are listed in sorted order for comparison. SQL guarantees an output order only with ORDER BY.

A priority change with ROLLBACK

```
SELECT ticket_id, priority
FROM metro_support.tickets WHERE ticket_id = 1002;
BEGIN;
UPDATE metro_support.tickets
SET priority = 'high'
WHERE ticket_id = 1002
RETURNING ticket_id, priority;
SELECT ticket_id, priority
FROM metro_support.tickets WHERE ticket_id = 1002;
ROLLBACK;
SELECT ticket_id, priority
FROM metro_support.tickets WHERE ticket_id = 1002;
```

Safe rehearsal

- Preview ticket 1002: medium. Run the whole block together.
- RETURNING and the next SELECT show high inside the transaction.
- After ROLLBACK, priority is medium again.

Checking a report independently

1. List the ticket IDs behind one staff count. Compare that list with the summary.
2. Keep a staff member with zero matches. COUNT(ticket_id) must return zero.
3. Check that tickets 1004 and 1009 still appear on an all-ticket dashboard.
4. A correct result on this dataset does not measure performance on a large service.

Lab 2: Repair the staff dashboard

One individual SQL file containing the repaired queries and brief comments.

- Keep all 12 tickets, including the two without assignees.
- Count active work for each staff member, including zero. Check one count directly.
- Test the priority change for ticket 1006, then roll it back to medium.

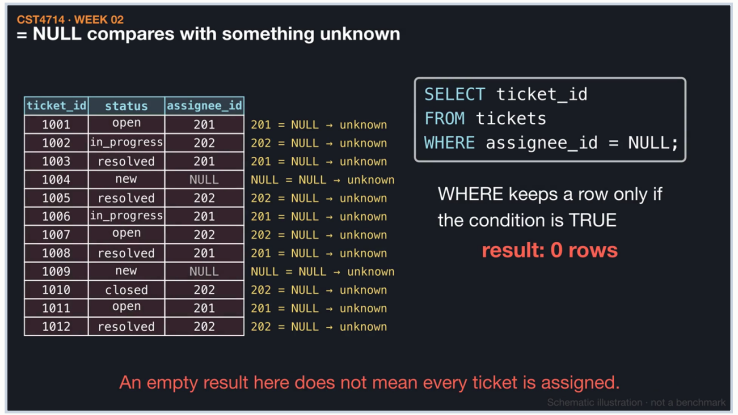
Brightspace submission: week_02_relational_sql_studio.sql

SQL review: results, relationships, and changes

Filters choose rows. Joins form pairs. Grouping changes what one result row represents.

- A missing assignee must not remove a ticket from an all-ticket report.
- COUNT(ticket_id) gives zero when a left join has no matching ticket.
- A targeted change can be inspected before ROLLBACK discards it.

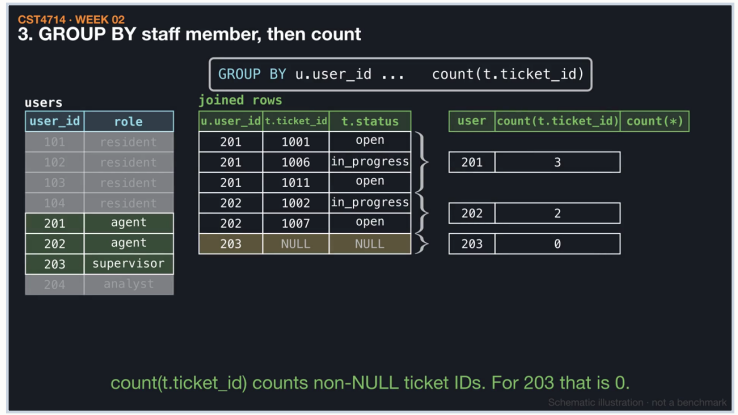
This week's videos



NULL: why = NULL finds nothing (1:20)



What does one row mean after a join? (1:16)



Counting workload, including zero (1:28)

Click a thumbnail to play on GitHub. Silent, with captions and .srt subtitles; diagrams are schematic.